

Exercice 1 : Formalisation ($\frac{4}{4}$ points)

Les variables propositionnelles N et T serviront, dans cet exercice, à représenter (respectivement) les propositions “Un étudiant a de bonnes notes” et “Un étudiant travaille”. À l’aide des variables propositionnelles N et T , formaliser les propositions suivantes (si, pour l’une ou l’autre d’entre elles, la traduction vous paraît impossible, dites-le et expliquez pourquoi) :

1. C’est seulement si un étudiant travaille qu’il a de bonnes notes.
2. Un étudiant n’a de bonnes notes que s’il travaille.
3. Pour un étudiant, le travail est une condition nécessaire à l’obtention de bonnes notes.
4. Un étudiant a de mauvaises notes, à moins qu’il ne travaille.
5. Malgré son travail, un étudiant a de mauvaises notes.
6. Un étudiant travaille seulement s’il a de bonnes notes.
7. À quoi bon travailler, si c’est pour avoir de mauvaises notes ?
8. Un étudiant a de bonnes notes sauf s’il ne travaille pas.

(0,5 point par réponse correcte)

1. C’est seulement si un étudiant travaille qu’il a de bonnes notes : $N \Rightarrow T$. La réponse $T \Leftrightarrow N$ est aussi acceptée.
2. Un étudiant n’a de bonnes notes que s’il travaille : $N \Leftrightarrow T$. La réponse $N \Rightarrow T$ est aussi acceptée.
3. Pour un étudiant, le travail est une condition nécessaire à l’obtention de bonnes notes : $N \Rightarrow T$.
4. Un étudiant a de mauvaises notes, à moins qu’il ne travaille : $(\neg N \wedge \neg T) \vee (T \wedge N)$, autrement dit $N \Leftrightarrow T$. La réponse $\neg N \vee (T \wedge N)$, c’est-à-dire $N \Rightarrow T$ est aussi acceptée.
5. Malgré son travail, un étudiant a de mauvaises notes : $T \wedge \neg N$.
6. Un étudiant travaille seulement s’il a de bonnes notes : $T \Rightarrow N$.
7. À quoi bon travailler, si c’est pour avoir de mauvaises notes ? : ceci n’est pas une proposition, mais une question, donc ne peut pas être formalisé par une proposition. On peut aussi interpréter cette question comme l’affirmation suivante : “On a des mauvaises notes, que l’on travaille ou pas”, qui se formaliserait par $\neg N$, autre réponse considérée comme correcte.
8. Un étudiant a de bonnes notes sauf s’il ne travaille pas : $N \Leftrightarrow T$. On accepte aussi $N \vee (\neg T \wedge \neg N)$, c’est-à-dire $T \Rightarrow N$.

Exercice 2 : Logique avec Why3 ($\frac{2}{2}$ points)

Donner un code complet WhyML permettant de vérifier avec l’interface graphique de Why3 que les propositions “ a implique b ” et “soit non a , soit a et b ” sont équivalentes, quelles que soient les propositions a et b .

(2 points : 1 point pour la déclaration des prédicats, 1 point pour le lemme)

```
predicate a
predicate b
lemma equiv: not a ∨ a ∧ b ↔ (a → b)
```

La réponse suivante est aussi acceptée, même si dans ce lemme `a` et `b` ne sont pas des prédicats mais des booléens.

```
lemma equiv: forall a b. not a ∨ a ∧ b ↔ (a → b)
```

Exercice 3 : Logique du premier ordre avec Why3 (3 points)

Un prédicat binaire p est dit irréflexif si $p(x, x)$ est toujours faux. Le code WhyML suivant déclare un prédicat binaire et la propriété d'irréflexivité pour ce prédicat, dans un prédicat WhyML nommé `irrefl`.

```
type t
predicate p t t
predicate irrefl = not (exists x. p x x)
```

Question 1 : Un prédicat binaire p est dit transitif si $p(x, y)$ et $p(y, z)$ impliquent $p(x, z)$.

Sous la forme d'un prédicat nommé, définir en WhyML la propriété de transitivité pour le prédicat binaire `p`.

Question 2 : Un prédicat binaire p est dit asymétrique si $p(x, y)$ implique que $p(y, x)$ est faux.

Sous la forme d'un prédicat nommé, définir en WhyML la propriété d'asymétrie pour le prédicat binaire `p`.

Question 3 : En utilisant ces prédicats nommés, définir un lemme WhyML permettant de démontrer avec l'interface en ligne de Why3 que tout prédicat binaire irréflexif et transitif est aussi asymétrique.

(3 points : 1 point par ligne de code correcte, 0 point si `exists` incorrect ou s'il n'y a pas de `forall`)

```
predicate trans = forall x. forall y. forall z. (p x y ∧ p y z) → p x z
predicate assym = forall x. forall y. p x y → not p y x
lemma ita : irrefl ∧ trans → assym
```

Exercice 4 : Lemmes et preuves sur la longueur d'une liste (7 points)

Dans cet exercice, on considère un type $\mathbb{Z}$ pour l'ensemble infini des entiers relatifs. Ce type est noté `int` en WhyML.

1. Reproduire les égalités suivantes et compléter les pointillés afin d'obtenir une définition correcte d'une fonction $length : liste(\alpha) \rightarrow \mathbb{Z}$ qui calcule la longueur d'une liste quelconque.

$$length(Nil) = \dots \quad (1)$$

$$length(Cons(a, l)) = \dots \quad (2)$$

(1 point) La fonction $length : liste(\alpha) \rightarrow \mathbb{N}$ est définie par les égalités suivantes :

$$length(Nil) = 0 \quad (3)$$

$$length(Cons(a, l)) = 1 + length(l) \quad (4)$$

2. Avec cette définition, rédiger une preuve inductive détaillée de la propriété que la longueur d'une liste l est toujours positive ou nulle, en indiquant le numéro de chaque égalité utilisée.

(2 points) Cas de base : $l = Nil$.

$$\begin{aligned} length(Nil) &= 0 \\ &\geq 0 \end{aligned} \quad \text{par (3)}$$

Induction : Supposons $length(l) \geq 0$. Alors

$$\begin{aligned} length(Cons(a, l)) &= 1 + length(l) \\ &\geq 1 + 0 \\ &\geq 0 \end{aligned} \quad \begin{array}{l} \text{par (4)} \\ \text{par l'hypothèse d'induction} \end{array}$$

3. Reproduire le code WhyML suivant et compléter les pointillés afin d'obtenir une définition en WhyML d'une fonction nommée `len` qui calcule la longueur d'une liste WhyML.

```
let rec function len (l: list 'a) : int
= match l with
| Nil → ...
| Cons ... → ...
end
```

(1 point)

```
let rec function len (l: list 'a) : int
= match l with
| Nil → 0
| Cons _ r → 1 + len r
end
```

4. Définir selon la syntaxe du langage WhyML un lemme qui formalise que la seule liste de longueur nulle est la liste `Nil`. On ne demande pas de démonstration de ce lemme.

(1 point si complet ; sinon 0,5 point si $\rightarrow$ au lieu de $\leftrightarrow$ ou s'il manque la partie `forall`)

```
lemma lenOnil: forall l: list 'a. len l = 0  $\leftrightarrow$  l = Nil
```

5. En utilisant la fonction `len`, exprimer sous la forme d'un lemme WhyML la propriété que la longueur d'une liste est toujours positive ou nulle.

(1 point si complet, 0 point s'il n'y a pas de `forall`)

```
lemma lengNonnegative: forall l: list 'a. len l  $\geq$  0
```

- Détailler clairement et simplement ce qu'il faut spécifier en WhyML pour obtenir une preuve automatique de ce lemme avec Why3.

(1 point) Pour prouver ce lemme automatiquement, il suffit d'ajouter la postcondition

`ensures { result ≥ 0 }`

Why3 prouve cette postcondition automatiquement et l'utilise pour prouver automatiquement ce lemme.

Exercice 5 : Indice d'une valeur maximale dans un tableau (*4 points + 1 point bonus*)

L'objectif de cet exercice est de spécifier et prouver automatiquement une propriété de la fonction du listing 1, qui calcule l'indice d'une valeur maximale dans le tableau `t`, qui est de longueur non nulle.

```
let max (t : array int) : int
  requires { 0 < length t }
= let n = length t in
  let ref imax = n-1 in
  let ref k = imax in
  while 0 ≤ k do
    if t[imax] < t[k] then imax := k;
    k := k-1
  done;
  imax
```

Listing 1 – Calcul de l'indice d'une valeur maximale dans un tableau.

- Proposer une postcondition en WhyML qui donne les bornes les plus précises possibles pour la valeur retournée par cette fonction.

(1 point si correct; que 0,5 point si `imax` au lieu de `result`; -0,5 point si `n` au lieu de `length t`)

`ensures { 0 ≤ result < length t }` (* Q1: 1 pt *)

- Proposer une postcondition en WhyML pour spécifier que cette fonction calcule l'indice d'une valeur maximale dans le tableau `t`.

(1 point, dont 0,5 point pour la condition sur `j`)

`ensures { forall j. 0 ≤ j < length t → t[result] ≥ t[j] }` (* Q2: 1 pt *)

- Proposer des invariants et un variant de boucle pour prouver ce contrat et la terminaison de la fonction.

(dans le premier invariant, 0,5 point pour la borne pour `k` et 0,5 point pour les bornes pour `imax`; 0 si $0 \leq k$ car ce n'est pas un invariant; 0,5 point si $k \leq j$ au lieu de $k < j$ dans le deuxième invariant, même si c'est faux)

```
invariant { k < n ∧ 0 ≤ imax < n } (* Q3: 1 pt *)
invariant { forall j. k < j < n → t[imax] ≥ t[j] } (* Q3: 1 pt *)
variant { k } (* Q3: 1 pt *)
```